

DBMS Practical Guide: Self-Contained Chits

Every chit below contains the exact terminal login, database creation, table setup, and final queries needed from a blank screen.

Chit 1: MySQL Joins & Subqueries

QUESTIONS

1. Show info for saving account. 2. Natural, left, right join. 3. Same city as 'pooja'. 4. Less than average amount. 5. Max amount in account. 6. Min amount of that Acc_type. 7. Higher than any saving amount.

1. TERMINAL LOGIN & DATABASE SETUP

```
sudo mysql -u root -p
CREATE DATABASE exam_db; USE exam_db;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Customer(C_Id INT, Cname VARCHAR(20), City VARCHAR(20));
CREATE TABLE Account(C_Id INT, Acc_Type VARCHAR(20), Amount INT);
INSERT INTO Customer VALUES (1,'John','Nashik'),(2,'Seema','Aurangabad'),(3,'Amita','Nagar'),(4,'Rakesh','Pune'),
(6,'Ankita','Chandwad'),(7,'Bhavika','Pune'),(8,'Deepa','Mumbai'),(9,'Nitin','Nagpur'),(10,'Pooja','Pune');
INSERT INTO Account VALUES (1,'Current',5000),(2,'Saving',20000),(3,'Saving',70000),(4,'Saving',50000),(6,'Current',35000),
(7,'Loan',30000),(8,'Saving',50000),(9,'Saving',90000),(10,'Loan',8000),(11,'Current',45000);
```

3. EXECUTION QUERIES

```
SELECT c.Cname, a.Acc_Type, a.Amount FROM Customer c JOIN Account a ON c.C_Id = a.C_Id WHERE a.Acc_Type='Saving';
SELECT * FROM Customer NATURAL JOIN Account;
SELECT * FROM Customer LEFT JOIN Account ON Customer.C_Id = Account.C_Id;
SELECT * FROM Customer RIGHT JOIN Account ON Customer.C_Id = Account.C_Id;
SELECT * FROM Customer WHERE City = (SELECT City FROM Customer WHERE Cname='Pooja');
SELECT * FROM Account WHERE Amount < (SELECT AVG(Amount) FROM Account);
SELECT C_Id FROM Account WHERE Amount = (SELECT MAX(Amount) FROM Account);
SELECT Amount, Acc_Type FROM Account a1 WHERE Amount=(SELECT MIN(Amount) FROM Account a2 WHERE a1.Acc_Type=a2.Acc_Type);
SELECT Amount FROM Account WHERE Amount > ANY (SELECT Amount FROM Account WHERE Acc_Type='Saving');
```

Chit 2: PL/SQL Control Structure (Library Fine)

QUESTIONS

Accept roll_no & book. Days 15-30: Rs 5/day. Days >30: Rs 50/day (>30) + Rs 5/day (<30). Change status I to R. Store in fine table.

1. TERMINAL LOGIN & SETUP

```
sqlplus username/password
SET SERVEROUTPUT ON;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Borrower(Rollin INT, Name VARCHAR2(20), Dateofissue DATE, NameofBook VARCHAR2(20), Status CHAR(1));
CREATE TABLE Fine(Roll_no INT, FineDate DATE, Amt INT);
INSERT INTO Borrower VALUES (1, 'Amit', SYSDATE - 35, 'DBMS Guide', 'I');
```

3. PL/SQL CODE BLOCK (TYPE / AND HIT ENTER TO RUN)

```
DECLARE
    v_days NUMBER; v_fine NUMBER := 0; v_roll NUMBER := &roll_no; v_book VARCHAR2(50) := '&book_name';
BEGIN
    v_days := &days_since_issue;
    IF v_days BETWEEN 15 AND 30 THEN v_fine := v_days * 5;
    ELSIF v_days > 30 THEN v_fine := (30 * 5) + ((v_days - 30) * 50); END IF;
    UPDATE Borrower SET Status = 'R' WHERE Rollin = v_roll AND NameofBook = v_book;
    IF v_fine > 0 THEN INSERT INTO Fine VALUES (v_roll, SYSDATE, v_fine); END IF;
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error Occurred');
END;
/
```

Chit 3: PL/SQL Exception Handling (Attendance)

QUESTIONS

Check attendance in Stud table. If < 75%: display "Term not granted", set status "D". Else: "Term granted", set "ND". Handle exceptions.

1. TERMINAL LOGIN & SETUP

```
sqlplus username/password
SET SERVEROUTPUT ON;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Stud(Roll INT, Att INT, Status VARCHAR2(5));
INSERT INTO Stud VALUES (1, 80, 'ND');
INSERT INTO Stud VALUES (2, 60, 'ND');
```

3. PL/SQL CODE BLOCK (TYPE / AND HIT ENTER TO RUN)

```
DECLARE
  v_roll NUMBER := &roll_no; v_att NUMBER;
BEGIN
  SELECT Att INTO v_att FROM Stud WHERE Roll = v_roll;
  IF v_att < 75 THEN
    DBMS_OUTPUT.PUT_LINE('Term not granted'); UPDATE Stud SET Status = 'D' WHERE Roll = v_roll;
  ELSE
    DBMS_OUTPUT.PUT_LINE('Term granted'); UPDATE Stud SET Status = 'ND' WHERE Roll = v_roll;
  END IF;
EXCEPTION WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Student not found');
END;
/
```

Chit 4: MySQL University Schema

QUESTIONS

i. Avg salary where dept avg > 42000. ii. Increase computer dept salary 10%. iii. Names neither 'Amol' nor 'Amit'. iv. Names containing 'am'. v. Comp dept students taking "DBMS".

1. TERMINAL LOGIN & DATABASE SETUP

```
sudo mysql -u root -p
CREATE DATABASE exam_db; USE exam_db;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE student(S_ID INT, name VARCHAR(20), dept_name VARCHAR(20), tot_cred INT);
CREATE TABLE instructor(T_ID INT, name VARCHAR(20), dept_name VARCHAR(20), salary INT);
CREATE TABLE course(course_id INT, title VARCHAR(20), dept_name VARCHAR(20), credits INT);
CREATE TABLE takes(S_ID INT, course_id INT);
INSERT INTO instructor VALUES (1,'Amol','Computer',40000),(2,'Amit','IT',50000),(3,'Rahul','Computer',45000);
INSERT INTO student VALUES (1,'Sam','Computer',40),(2,'Ramesh','IT',30);
INSERT INTO course VALUES (101,'DBMS','Computer',4);
INSERT INTO takes VALUES (1, 101);
```

3. EXECUTION QUERIES

```
SELECT dept_name, AVG(salary) FROM instructor GROUP BY dept_name HAVING AVG(salary) > 42000;
UPDATE instructor SET salary = salary + (salary * 0.10) WHERE dept_name = 'Computer';
SELECT name FROM instructor WHERE name NOT IN ('Amol', 'Amit');
SELECT name FROM student WHERE name LIKE '%am%';
SELECT s.name FROM student s JOIN takes t ON s.S_ID=t.S_ID JOIN course c ON t.course_id=c.course_id WHERE
s.dept_name='Computer' AND c.title='DBMS';
```

Chit 5: PL/SQL Cursors (RollCall Merge)

QUESTIONS

Merge data from N_RollCall to O_RollCall using parameterized cursor. Skip if data already exists in O_RollCall.

1. TERMINAL LOGIN & SETUP

```
sqlplus username/password
SET SERVEROUTPUT ON;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE N_RollCall(Roll INT, Name VARCHAR2(20), Status VARCHAR2(10));
CREATE TABLE O_RollCall(Roll INT, Name VARCHAR2(20), Status VARCHAR2(10));
INSERT INTO N_RollCall VALUES (1, 'Alice', 'Present'); INSERT INTO N_RollCall VALUES (2, 'Bob', 'Absent');
INSERT INTO O_RollCall VALUES (1, 'Alice', 'Present');
```

3. PL/SQL CODE BLOCK (TYPE / AND HIT ENTER TO RUN)

```
DECLARE
CURSOR c_merge IS SELECT * FROM N_RollCall; v_exists NUMBER;
BEGIN
FOR r IN c_merge LOOP
SELECT COUNT(*) INTO v_exists FROM O_RollCall WHERE Roll = r.Roll;
IF v_exists = 0 THEN INSERT INTO O_RollCall VALUES (r.Roll, r.Name, r.Status); END IF;
END LOOP;
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error');
END;
/
```

Chit 6: PL/SQL Stored Procedure (Grades)

QUESTIONS

Proc proc_Grade: <=1500 & >=990 Distinction, 989-900 First, 899-825 Higher Second. Write block to use it inserting into Result table.

1. TERMINAL LOGIN & SETUP

```
sqlplus username/password
SET SERVEROUTPUT ON;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Stud_Marks(name VARCHAR2(20), total_marks NUMBER);
CREATE TABLE Result(Roll NUMBER, Name VARCHAR2(20), Class VARCHAR2(30));
INSERT INTO Stud_Marks VALUES ('Alice', 950);
```

3. PL/SQL CODE BLOCK (CREATE PROCEDURE, THEN EXECUTE)

```
CREATE OR REPLACE PROCEDURE proc_Grade (p_roll IN NUMBER, p_name IN VARCHAR2, p_marks IN NUMBER) IS
v_class VARCHAR2(20);
BEGIN
IF p_marks <= 1500 AND p_marks >= 990 THEN v_class := 'Distinction';
ELSIF p_marks <= 989 AND p_marks >= 900 THEN v_class := 'First Class';
ELSIF p_marks <= 899 AND p_marks >= 825 THEN v_class := 'Higher Second Class';
ELSE v_class := 'Other'; END IF;
INSERT INTO Result VALUES (p_roll, p_name, v_class);
END;
/
-- Calling block:
BEGIN proc_Grade(1, 'Alice', 950); END;
/
```

Chit 7: PL/SQL Stored Function (Grades)

QUESTIONS

Function proc_Grade (same logic as Procedure). Write block to use it.

1. TERMINAL LOGIN & SETUP

```
sqlplus username/password
SET SERVEROUTPUT ON;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Stud_Marks(name VARCHAR2(20), total_marks NUMBER);
CREATE TABLE Result(Roll NUMBER, Name VARCHAR2(20), Class VARCHAR2(30));
```

3. PL/SQL CODE BLOCK (CREATE FUNCTION, THEN EXECUTE)

```
CREATE OR REPLACE FUNCTION func_Grade (p_marks IN NUMBER) RETURN VARCHAR2 IS
BEGIN
  IF p_marks <= 1500 AND p_marks >= 990 THEN RETURN 'Distinction';
  ELSIF p_marks <= 989 AND p_marks >= 900 THEN RETURN 'First Class';
  ELSIF p_marks <= 899 AND p_marks >= 825 THEN RETURN 'Higher Second Class';
  ELSE RETURN 'Other'; END IF;
END;
/
-- Calling block:
DECLARE v_res VARCHAR2(30);
BEGIN v_res := func_Grade(950); DBMS_OUTPUT.PUT_LINE('Grade is: ' || v_res); END;
/
```

Chit 8: MySQL University Views

QUESTIONS

i. Instructor names taught semester wise. ii. View retrieving student details. iii. Rename dept_name to deptatrmnt_name. iv. Delete student if dept is NULL.

1. TERMINAL LOGIN & DATABASE SETUP

```
sudo mysql -u root -p
CREATE DATABASE exam_db; USE exam_db;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE student(S_ID INT, name VARCHAR(20), dept_name VARCHAR(20), tot_cred INT);
CREATE TABLE instructor(T_ID INT, name VARCHAR(20), dept_name VARCHAR(20), salary INT);
CREATE TABLE course(course_id INT, title VARCHAR(20), dept_name VARCHAR(20), credits INT);
CREATE TABLE teaches(T_ID INT, course_id INT, sec_id INT, semester VARCHAR(10), year INT);
INSERT INTO instructor VALUES (1,'John','Comp',50000);
INSERT INTO teaches VALUES (1,101,1,'Fall',2023);
INSERT INTO student VALUES (1,'Alice','Comp',40), (2,'Bob',NULL,30);
```

3. EXECUTION QUERIES

```
SELECT DISTINCT i.name, t.semester FROM instructor i JOIN teaches t ON i.T_ID = t.T_ID;
CREATE VIEW Student_View AS SELECT S_ID, name, dept_name FROM student;
ALTER TABLE student RENAME COLUMN dept_name TO department_name;
DELETE FROM student WHERE department_name IS NULL;
```

Chit 9: MongoDB Orderinfo Index/Agg

QUESTIONS

i. Avg price status 'A'. ii. Status of amt 100-1000. iii. Info without _id. iv. Simple index and fire queries.

1. TERMINAL LOGIN & DATABASE SETUP

```
mongosh
use dbms_exam;
```

2. COLLECTION CREATION & DATA INSERTION

```
db.orderinfo.insertMany([
  {cust_id:123, cust_name:"abc", status:"A", price:250},
  {cust_id:124, cust_name:"xyz", status:"B", price:500},
  {cust_id:125, cust_name:"pqr", status:"A", price:800}
]);
```

3. EXECUTION QUERIES

```
db.orderinfo.aggregate([{$match: {status: "A"}}, {$group: {_id: "$cust_id", avgPrice: {$avg: "$price"}}}]);
db.orderinfo.find({price: {$gte: 100, $lte: 1000}}, {status: 1, _id: 0});
db.orderinfo.find({}, {_id: 0});
db.orderinfo.createIndex({cust_id: 1});
db.orderinfo.find({cust_id: 123}).explain("executionStats");
```

Chit 10 & 11: PL/SQL Triggers (Library Audit)

QUESTIONS

Chit 10: Before Trigger | Chit 11: After Trigger. Keep track of updated/deleted records. Old value into Library_Audit.

1. TERMINAL LOGIN & SETUP

```
sqlplus username/password
SET SERVEROUTPUT ON;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Library(BookID INT, BookName VARCHAR2(20), Status CHAR(1));
CREATE TABLE Library_Audit(BookID INT, BookName VARCHAR2(20), Status CHAR(1), AuditDate DATE);
INSERT INTO Library VALUES (101, 'Oracle Guide', 'A');
```

3. PL/SQL CODE BLOCK (CREATE TRIGGER, THEN TEST)

```
-- FOR CHIT 10 (BEFORE TRIGGER):
CREATE OR REPLACE TRIGGER trg_lib_before
BEFORE UPDATE OR DELETE ON Library FOR EACH ROW
BEGIN INSERT INTO Library_Audit VALUES (:OLD.BookID, :OLD.BookName, :OLD.Status, SYSDATE); END;
/

-- FOR CHIT 11 (AFTER TRIGGER):
CREATE OR REPLACE TRIGGER trg_lib_after
AFTER UPDATE OR DELETE ON Library FOR EACH ROW
BEGIN INSERT INTO Library_Audit VALUES (:OLD.BookID, :OLD.BookName, :OLD.Status, SYSDATE); END;
/

-- Test the trigger (Run this to show it works):
UPDATE Library SET Status = 'I' WHERE BookID = 101;
SELECT * FROM Library_Audit;
```

Chit 12: MongoDB Movies

QUESTIONS

i. Name budget > 100000. ii. Producer in Pune. iii. Update action to horror. iv. Producer1 docs with address.

1. TERMINAL LOGIN & DATABASE SETUP

```
mongosh
use dbms_exam;
```

2. COLLECTION CREATION & DATA INSERTION

```
db.movies.insertMany([
  {name: "Movie1", type: "action", budget: 1000000, producer:{name: "producer1", address: "PUNE"}},
  {name: "Movie2", type: "comedy", budget: 50000, producer:{name: "producer2", address: "MUMBAI"}}
]);
```

3. EXECUTION QUERIES

```
db.movies.find({budget: {$gt: 100000}}, {name: 1, _id: 0});
db.movies.find({"producer.address": "PUNE"}, {"producer.name": 1, _id: 0});
db.movies.updateMany({type: "action"}, {$set: {type: "horror"}});
db.movies.find({"producer.name": "producer1"}, {"producer.address": 1});
```

Chit 13: MongoDB Teachers & Students

QUESTIONS

1. Insert docs. 2. Teachers Dno=2, salary>=10000. 3. Student Roll=2 or Sname='xyz'. 4. Update Roll=5. 5. Delete Class='FE'. 6. Apply index.

1. TERMINAL LOGIN & DATABASE SETUP

```
mongosh
use dbms_exam;
```

2. COLLECTION CREATION & DATA INSERTION

```
db.Teachers.insertMany([{Tname:"A", dno:2, Experience:5, Salary:15000, Data_of_Joining:"2020-01"}]);
db.Department.insertMany([{Dno:2, Dname:"Comp"}]);
db.Students.insertMany([{Sname:"xyz", Roll_No:2, Class:"SE"}, {Sname:"abc", Roll_No:5, Class:"FE"}]);
```

3. EXECUTION QUERIES

```
db.Teachers.find({dno: 2, Salary: {$gte: 10000}});
db.Students.find({$or: [{Roll_No: 2}, {Sname: "xyz"}]});
db.Students.updateOne({Roll_No: 5}, {$set: {Sname: "NewName"}});
db.Students.deleteMany({Class: "FE"});
db.Students.createIndex({Roll_No: 1});
```

Chit 14: MongoDB Institute

QUESTIONS

1-3 Create/Insert. 4. Disp all. 5. Age>15. 6. Sort name. 7. Update branch Comp Roll 3. 8. Remove Roll 1. 9. Name starts A. 10. Total docs. 11. First 5. 12. Skip 3. 13. Live in Pune. 14. Only name. 15. Drop.

1. TERMINAL LOGIN & DATABASE SETUP

```
mongosh
```

2. COLLECTION CREATION & DATA INSERTION

```
use Institute;
db.createCollection("Students");
db.Students.insertMany([
  {RollNo:1, StudentName:"Amit", Age:20, Branch:"IT", Address:{City:"Pune", State:"MH"}, Hobbies:["Reading"]},
  {RollNo:3, StudentName:"Rahul", Age:14, Branch:"Mech", Address:{City:"Mumbai", State:"MH"}, Hobbies:["Cricket"]},
  {RollNo:4, StudentName:"Aarti", Age:18, Branch:"Civil", Address:{City:"Pune", State:"MH"}, Hobbies:["Art"]}
]);
```

3. EXECUTION QUERIES

```
db.Students.find();
db.Students.find({Age: {$gt: 15}});
db.Students.find().sort({StudentName: 1});
db.Students.updateOne({RollNo: 3}, {$set: {Branch: "Computer"}});
db.Students.deleteOne({RollNo: 1});
db.Students.find({StudentName: /^A/});
db.Students.countDocuments();
db.Students.find().limit(5);
db.Students.find().skip(3);
db.Students.find({"Address.City": "Pune"}, {StudentName: 1, _id: 0});
db.Students.find({}, {StudentName: 1, _id: 0});
db.Students.drop();
```

Chit 15: MongoDB Aggregation (Department)

QUESTIONS

1-2 Create/Insert. 3. Dept avg sal. 4. Emp count each dept. 5. Dept min sal. 6. Apply/drop index.

1. TERMINAL LOGIN & DATABASE SETUP

```
mongosh
```

2. COLLECTION CREATION & DATA INSERTION

```
use department;
db.createCollection("teacher");
db.teacher.insertMany([
  {name:"X", department:"Comp", experience:5, salary:50000},
  {name:"Y", department:"IT", experience:3, salary:40000},
  {name:"Z", department:"Comp", experience:8, salary:80000}
]);
```

3. EXECUTION QUERIES

```
db.teacher.aggregate([{$group: {_id: "$department", avgSal: {$avg: "$salary"}}}]);
db.teacher.aggregate([{$group: {_id: "$department", count: {$sum: 1}}}]);
db.teacher.aggregate([{$group: {_id: "$department", minSal: {$min: "$salary"}}}]);
db.teacher.createIndex({name: 1});
db.teacher.dropIndex({name: 1});
```

Chit 16: MySQL Employee & Project

QUESTIONS

1. Diff locs. 2. Max/min sal. 3. Asc sal. 4. Nasik/Pune. 5. No comm. 6. Amit to Nashik. 7. Starts 'A'. 8. Count Mumbai. 9. Count each city. 10. Cross Addr. 11. City min sal. 12. City max sal > 26k. 13. Delete sal > 30k.

1. TERMINAL LOGIN & DATABASE SETUP

```
sudo mysql -u root -p
CREATE DATABASE exam_db; USE exam_db;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Employee(Eid INT, EName VARCHAR(20), Address VARCHAR(20), Salary INT, Commision INT);
CREATE TABLE Project(PrNo INT, Addr VARCHAR(20));
INSERT INTO Employee VALUES (1,'Amit','Pune',35000,5000),(2,'Sneha','Pune',25000,NULL),(3,'Savita','Nasik',28000,2000),
(4,'Pooja','Mumbai',19000,NULL),(5,'Sagar','Mumbai',25000,3000);
INSERT INTO Project VALUES (10,'Mumbai'),(20,'Pune'),(30,'Jalgoan');
```

3. EXECUTION QUERIES

```
SELECT DISTINCT Address FROM Employee;
SELECT MAX(Salary), MIN(Salary) FROM Employee;
SELECT * FROM Employee ORDER BY Salary ASC;
SELECT EName FROM Employee WHERE Address IN ('Nasik', 'Pune');
SELECT EName FROM Employee WHERE Commision IS NULL OR Commision = 0;
UPDATE Employee SET Address = 'Nashik' WHERE EName = 'Amit';
SELECT * FROM Employee WHERE EName LIKE 'A%';
SELECT COUNT(*) FROM Employee WHERE Address = 'Mumbai';
SELECT Address, COUNT(*) FROM Employee GROUP BY Address;
SELECT DISTINCT e.Address, p.Addr FROM Employee e CROSS JOIN Project p;
SELECT Address, MIN(Salary) FROM Employee GROUP BY Address;
SELECT Address, MAX(Salary) FROM Employee GROUP BY Address HAVING MAX(Salary) > 26000;
DELETE FROM Employee WHERE Salary > 30000;
```

Chit 17: MySQL Sequence, Constraints & View

QUESTIONS

Create emp table pk auto-increment from 101, default address. Add Post. Insert data. Create index Ename. Create view Ename, Salary.

1. TERMINAL LOGIN & DATABASE SETUP

```
sudo mysql -u root -p
CREATE DATABASE exam_db; USE exam_db;
```

2. TABLE CREATION & EXECUTION (DONE SEQUENTIALLY)

```
CREATE TABLE emp (
  Eno INT PRIMARY KEY AUTO_INCREMENT, Ename VARCHAR(50) NOT NULL,
  Address VARCHAR(50) DEFAULT 'Nashik', Joindate DATE
) AUTO_INCREMENT=101;
ALTER TABLE emp ADD Post VARCHAR(50);
INSERT INTO emp (Ename, Joindate, Post) VALUES ('Rahul', '2025-01-01', 'Developer');
CREATE INDEX idx_ename ON emp(Ename);
ALTER TABLE emp ADD Salary INT;
CREATE VIEW emp_view AS SELECT Ename, Salary FROM emp;
```

Chit 18: MongoDB Orderinfo Complex Index

QUESTIONS

i. Add Age. ii. Complex index, drop dups. iii. Avg price group status. iv. Change name status B.

1. TERMINAL LOGIN & DATABASE SETUP

```
mongosh
use dbms_exam;
```

2. COLLECTION CREATION & DATA INSERTION

```
db.orderinfo.insertMany([{"cust_id":123, "cust_name":"abc", "status":"A", "price":250}, {"cust_id":124, "cust_name":"xyz", "status":"B", "price":500}]);
```

3. EXECUTION QUERIES

```
db.orderinfo.updateMany({}, {$set: {Age: 25}});
db.orderinfo.createIndex({"cust_id": 1, "status": 1});
db.orderinfo.aggregate([{$group: {_id: "$status", avgPrice: {$avg: "$price"}}}]);
db.orderinfo.updateMany({"status": "B"}, {$set: {"cust_name": "Updated Name"}});
```

Chit 19: MySQL Company & Works

QUESTIONS

i. ABC to Pune. ii. Mgrs Mbank 10% raise, if >20k 3%. iii. Emp in Bosch Pune. iv. Delete SBC >50k.

1. TERMINAL LOGIN & DATABASE SETUP

```
sudo mysql -u root -p
CREATE DATABASE exam_db; USE exam_db;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Company(c_id INT, cname VARCHAR(20), city VARCHAR(20));
CREATE TABLE Manager(mgr_id INT, mgrname VARCHAR(20));
CREATE TABLE works(emp_id INT, c_id INT, ename VARCHAR(20), cname VARCHAR(20), sal INT);
INSERT INTO Company VALUES (1,'ABC','Mumbai'),(2,'Mbank','Pune'),(3,'Bosch','Pune'),(4,'SBC Company','Delhi');
INSERT INTO Manager VALUES (101,'BossMan');
INSERT INTO works VALUES (1,2,'John','Mbank',25000), (101,2,'BossMan','Mbank',60000), (2,3,'Jane','Bosch',30000),
(3,4,'Mike','SBC Company',60000);
```

3. EXECUTION QUERIES

```
UPDATE Company SET city = 'Pune' WHERE cname = 'ABC';
UPDATE works SET sal = CASE WHEN sal > 20000 THEN sal * 1.03 ELSE sal * 1.10 END WHERE cname = 'Mbank' AND emp_id IN (SELECT
mgr_id FROM Manager);
SELECT w.ename FROM works w JOIN Company c ON w.c_id = c.c_id WHERE c.cname = 'Bosch' AND c.city = 'Pune';
DELETE FROM works WHERE cname = 'SBC Company' AND sal > 50000;
```

Chit 20: MongoDB Student Aggregation Deep Dive

QUESTIONS

Create indices (single, compound, unique), show info, remove. Aggregation: max, min, sum, avg, first, last, count marks for each subject.

1. TERMINAL LOGIN & DATABASE SETUP

```
mongosh
use dbms_exam;
```

2. COLLECTION CREATION & DATA INSERTION

```
db.Student.insertMany([
  {Rollno:1, name:'Navin', subject:'DMSA', marks:78}, {Rollno:2, name:'anusha', subject:'OSD', marks:75},
  {Rollno:3, name:'ravi', subject:'TOC', marks:69}, {Rollno:4, name:'veena', subject:'TOC', marks:70},
  {Rollno:5, name:'Pravini', subject:'OSD', marks:80}, {Rollno:6, name:'Reena', subject:'DMSA', marks:50},
  {Rollno:7, name:'Geeta', subject:'CN', marks:90}, {Rollno:8, name:'Akash', subject:'CN', marks:85}
]);
```

3. EXECUTION QUERIES

```
db.Student.createIndex({Rollno: 1});
db.Student.createIndex({Rollno: 1, subject: 1});
db.Student.createIndex({Rollno: 1}, {unique: true});
db.Student.getIndexes();
db.Student.dropIndex({Rollno: 1});

db.Student.aggregate([{$group: {_id: "$subject", maxMarks: {$max: "$marks"}}}]);
db.Student.aggregate([{$group: {_id: "$subject", minMarks: {$min: "$marks"}}}]);
db.Student.aggregate([{$group: {_id: "$subject", sumMarks: {$sum: "$marks"}}}]);
db.Student.aggregate([{$group: {_id: "$subject", avgMarks: {$avg: "$marks"}}}]);
db.Student.aggregate([{$sort: {Rollno: 1}}, {$group: {_id: "$subject", firstRec: {$first: "$name"}}}]);
db.Student.aggregate([{$sort: {Rollno: 1}}, {$group: {_id: "$subject", lastRec: {$last: "$name"}}}]);
db.Student.aggregate([{$group: {_id: "$subject", count: {$sum: 1}}}]);
```

Chit 21: MySQL Duty Allocation

QUESTIONS

i. Duty e_no 123 first shift April 2003. ii. Pay >= Sachin. iii. View min, max, avg post. iv. Count shifts manager.

1. TERMINAL LOGIN & DATABASE SETUP

```
sudo mysql -u root -p
CREATE DATABASE exam_db; USE exam_db;
```

2. TABLE CREATION & DATA INSERTION

```
CREATE TABLE Empl(e_no INT, e_name VARCHAR(20), post VARCHAR(20), pay_rate INT);
CREATE TABLE Position(pos_no INT, post VARCHAR(20));
CREATE TABLE `Duty-alloc`(pos_no INT, e_no INT, month VARCHAR(15), year INT, shift VARCHAR(15));
INSERT INTO Empl VALUES (123,'Rahul','Worker',500),(124,'Sachin','Worker',600),(125,'Boss','manager',1000);
INSERT INTO Position VALUES (1,'Worker'),(2,'manager');
INSERT INTO `Duty-alloc` VALUES (1,123,'April',2003,'first'),(2,125,'April',2003,'first');
```

3. EXECUTION QUERIES

```
SELECT * FROM `Duty-alloc` WHERE e_no = 123 AND shift = 'first' AND month = 'April' AND year = 2003;
SELECT * FROM Empl WHERE pay_rate >= (SELECT pay_rate FROM Empl WHERE e_name = 'Sachin');
CREATE VIEW Post_Salary_Stats AS SELECT p.post, MIN(e.pay_rate), MAX(e.pay_rate), AVG(e.pay_rate)
FROM Position p JOIN Empl e ON p.pos_no = e.e_no GROUP BY p.post;
SELECT shift, COUNT(d.e_no) FROM `Duty-alloc` d JOIN Position p ON d.pos_no = p.pos_no WHERE p.post = 'manager' GROUP BY
shift;
```